

Universidad Católica de Temuco

Ingeniería Civil en Informática

INFORME DE TAREA

Análisis léxico del lenguaje Prolog

Teoría de la Computación

Dato	Información
Código curso	INFO1148
Integrantes	Fernando Alonso Valdés Gómez Juan Francisco Muñoz Veloso Vicente Rodrigo Rivera Sánchez
Profesor	Prof. M. Lévano
Fecha de entrega	21 de septiembre de 2026

Temuco, 2026

Resumen

Este informe presenta el análisis, diseño, implementación y validación de un analizador léxico para un subconjunto de Prolog inspirado en la notación de ISO Prolog y SWI-Prolog (ISO/IEC, 1995; SWI-Prolog, s. f.). El trabajo define formalmente el catálogo de tokens con una expresión regular por categoría, modela las categorías principales con autómatas finitos, muestra la determinización por construcción de subconjuntos y la minimización por refinamiento de particiones sobre un subconjunto representativo, e implementa el lexer en Python con recorrido de izquierda a derecha, máxima coincidencia y prioridades explícitas. El lexer emite tipo, lexema original, línea y columna por token, administra una tabla de lexemas con deduplicación por par tipo y lexema, y reporta seis clases de error con fragmento y posición continuando el análisis. La validación usa una suite de 86 pruebas automatizadas con 27 casos válidos, 22 de prioridad, 16 inválidos y dos archivos completos, con resultado real de 86 aprobadas y cero fallos.

Palabras clave: análisis léxico, Prolog, expresiones regulares, autómatas finitos, tabla de lexemas.

1. Introducción

El análisis léxico es la primera fase de un compilador: transforma la cadena fuente en una secuencia ordenada de tokens con posición, separando las decisiones regulares (qué formas son válidas) de las decisiones sintácticas (en qué orden pueden aparecer). Un lexer correcto y bien diagnosticado simplifica el parser, produce errores comprensibles y deja evidencia reproducible de cada decisión.

El trabajo se plantea como un recorrido de izquierda a derecha. En cada posición el lexer identifica la categoría que puede comenzar allí, busca el lexema válido más largo y aplica la prioridad definida cuando dos categorías empatan. Este orden conecta las expresiones regulares, los autómatas y la implementación: los diagramas explican los recorridos posibles, mientras que el código decide qué recorrido tiene precedencia cuando las formas se superponen.

El documento avanza desde el problema hacia la evidencia. Primero fija el alcance y el catálogo; después formaliza las expresiones regulares y los autómatas; a continuación describe la arquitectura, la tabla de lexemas y la recuperación de errores; finalmente presenta las pruebas reproducibles y las conclusiones. La separación entre especificación, implementación y validación permite localizar una regla en la especificación, seguirla en el código y comprobarla en una prueba.

2. Objetivos

2.1. Objetivo general

Analizar, diseñar, implementar y validar un analizador léxico para un subconjunto definido de Prolog, aplicando alfabetos, lenguajes regulares, expresiones regulares y autómatas finitos, con pruebas reproducibles y diagnósticos verificables.

2.2. Objetivos específicos

- Definir cada categoría con nombre de token, patrón, ejemplos válidos e inválidos y atributo asociado.
- Construir una expresión regular por categoría y resolver superposiciones con máxima coincidencia y prioridades explícitas.
- Modelar las categorías principales con AFN o AFD e identificar estados iniciales, de aceptación y de error.
- Mostrar la determinización por construcción de subconjuntos y la minimización con justificación de equivalencias sobre un subconjunto representativo.
- Implementar el lexer con posiciones 1-based, tabla de lexemas y seis clases de error recuperables.

- Validar con al menos 20 pruebas válidas, 8 inválidas, casos de prioridad y dos archivos completos.

3. Alcance del analizador

El analizador reconoce exactamente el catálogo de docs/informe/01_especificacion_lexica.md: átomos simples y entre comillas simples, variables nombradas y anónima, enteros y reales sin signo, cadenas con comillas dobles, operadores de las seis familias (cláusula, consulta, DCG, unificación y comparación, aritméticos y control), nueve delimitadores de un carácter y elementos ignorables (espacios, tabulaciones, saltos de línea y comentarios de línea y de bloque), según las categorías exigidas en el enunciado (Lévano, 2026).

Quedan fuera del alcance el análisis sintáctico y semántico: no se verifica el orden de los tokens, no se construye un árbol sintáctico, no se comprueba el balance de paréntesis, corchetes o llaves, no se resuelven ámbitos ni variables repetidas, no se implementa unificación, no se evalúan expresiones y no se decide si una secuencia de tokens forma un programa válido. Una `)` aislada es un token `RPAREN` correcto; su posible improcedencia pertenece a una fase posterior.

4. Especificación léxica

Token	Descripción	Expresión regular	Ejemplos válidos	Ejemplos inválidos	Atributo
ATOM	Minúscula inicial; continúa con letras, dígitos o guion bajo.	<code>[a-z] [A-Za-z0-9_]*</code>	padre, persona_1, personaX	Padre, átomo	índice <i>i</i>
QUOTED_ATOM	Comillas simples, escapes válidos, sin salto real.	<code>'(\\['\nrt] [^'\\n\r])*'</code>	'Juan Perez', ':-', 'it\'s'	'sin cierre, 'a\q'	índice <i>i</i>
VARIABLE	Mayúscula o guion bajo más continuación.	<code>[A-Z] [A-Za-z0-9_]* _ [A-Za-z0-9_]+</code>	X, Persona, _Tmp	9X, _ (es anónima)	índice <i>i</i>
ANONYMOUS_VARIABLE	Guion bajo aislado, con guarda de no-continuación.	<code>_ (?! [A-Za-z0-9_])</code>	_	_X, __ (son variables)	—
INTEGER	Dígitos ASCII sin signo.	<code>[0-9]+</code>	0, 25, 0007	-25, 5., 12abc	índice <i>i</i>
REAL	Dígitos, punto y dígitos a ambos lados.	<code>[0-9]+\.[0-9]+</code>	3.14, 0.5, 10.00	.5, 5., 1e3	índice <i>i</i>
STRING	Comillas dobles, escapes válidos, sin salto real.	<code>"(\\["\nrt] [^"\\n\r])*"</code>	"hola", "dice"	"sin cierre, "a\q"	índice <i>i</i>
OPERATOR	Operador exacto; se prueba por longitud descendente.	<code>--> \\== \\.\\. == \\= < > = :- \\?- // ** = < > / * \\+ - \\ \\+ ! ; is(?![A-Za-z0-9_]) mod(?![A-Za-z0-9_])</code>	:-, ?-, -->, =, ., //, mod	:-, ?, \\ (sola)	familia
Delimitadores	Nueve símbolos de un carácter, un token cada uno.	<code>\\(\\)\\[\\]\\{\\}\\ \\ , \\.</code>	() [] { } , .	@ (no es delimitador)	—
COMMENT	Línea % o bloque /* ... */; se ignora.	<code>%[\\r\\n]*, /*(?:[*] *(?!\\/))**/</code>	% Hechos, /* ... */	/* sin cierre	—
WHITESPACE	Espacios, tabulaciones y saltos; se ignora.	<code>[\\t\\r\\n]+</code>	espacio, tab, salto	\\v (no admitido)	—

Las familias de OPERATOR son cláusula (: -), consulta (? -), DCG (-->), unificación y comparación (=, \=, ==, \==, =. ., <, =<, >, >=), aritméticos (+, -, *, /, //, **, is, mod) y control (\+, !, ;). Las palabras is y mod se reconocen solo cuando el lexema completo no tiene continuación de identificador, es decir, con la guarda de frontera (?! [A-Za-z0-9_]); así, isla y modulo son átomos completos.

Los números no tienen signo; -12 son dos tokens. INTEGER y REAL no admiten exponentes ni puntos sin dígitos a ambos lados. Los escapes válidos son la barra con el delimitador, más \n, \r y \t. Ningún literal admite un salto de línea real. El signo % inicia un comentario de línea hasta antes del salto y /* abre un bloque que termina en el primer */ sin anidamiento. Las posiciones son 1-based; la tabulación avanza una columna y CRLF cuenta como un único salto. El lexema conservado es siempre el segmento original, sin normalización ni conversión. Las expresiones regulares de la tabla son las efectivas que acepta el programa y coinciden con la especificación contractual de docs/informe/01_especificacion_lexica.md (Equipo de trabajo, 2026). Las filas COMMENT y WHITESPACE documentan elementos que se reconocen y se ignoran: actualizan la posición (CRLF cuenta como un único salto) sin emitir token ni entrada de tabla.

5. Diseño de los autómatas

Las figuras usan la convención estado --símbolo--> estado; el doble círculo indica aceptación y el rótulo inferior el token emitido. Los alfabetos abreviados son LOWER=[a-z], LETTER=[A-Za-z], UPPER=[A-Z], DIGIT=[0-9] e ID=[A-Za-z0-9_].

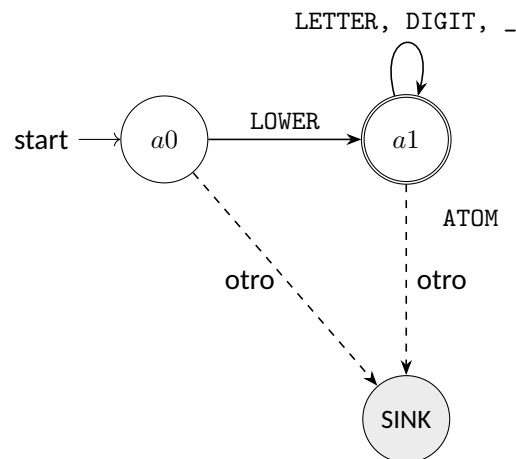


Figura 1: Autómata para átomos no entrecomillados.

El AFD de átomos reconoce $\text{LOWER (LETTER U DIGIT U \{ _ \})}^*$ con máxima coincidencia: $q_0 \xrightarrow{\text{LOWER}} q_1$ y $q_1 \xrightarrow{[A-Za-z0-9_]} q_1$, donde q_1 acepta ATOM.

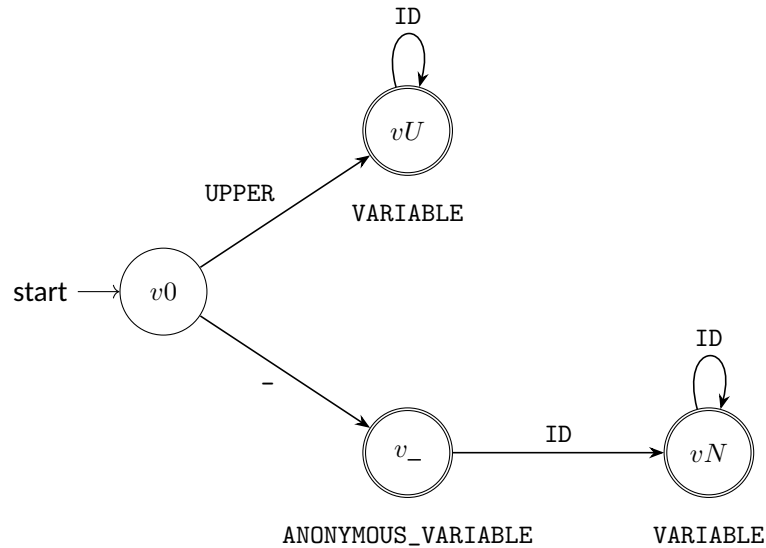


Figura 2: Autómata para variables y variable anónima.

El AFD de variables distingue la variable con nombre de la anónima: vU y vN aceptan `VARIABLE`, mientras que $v_$ sin continuación se resuelve como `ANONYMOUS_VARIABLE` por regla de longitud y prioridad.

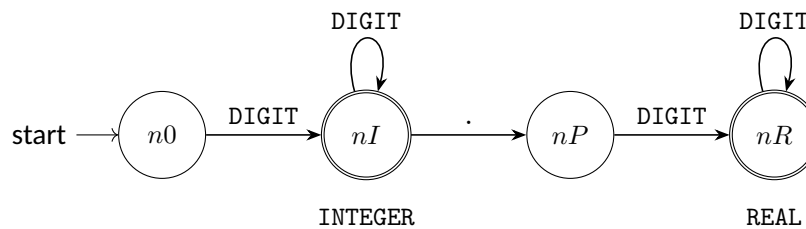


Figura 3: Autómata para números enteros y reales.

El AFD de números acepta en nI (`INTEGER`) y nR (`REAL`); nP no acepta, por lo que `5 .` no pertenece al lenguaje numérico y se diagnostica como número mal formado.

Los literales entrecomillados siguen el esquema `D . . . D` con estado de escape: el delimitador abre el literal, los caracteres seguros ciclan, la barra lleva al estado de escape y solo un escape válido o el delimitador de cierre permiten continuar. Un salto de línea real o un escape desconocido no tienen transición válida y producen el error correspondiente.

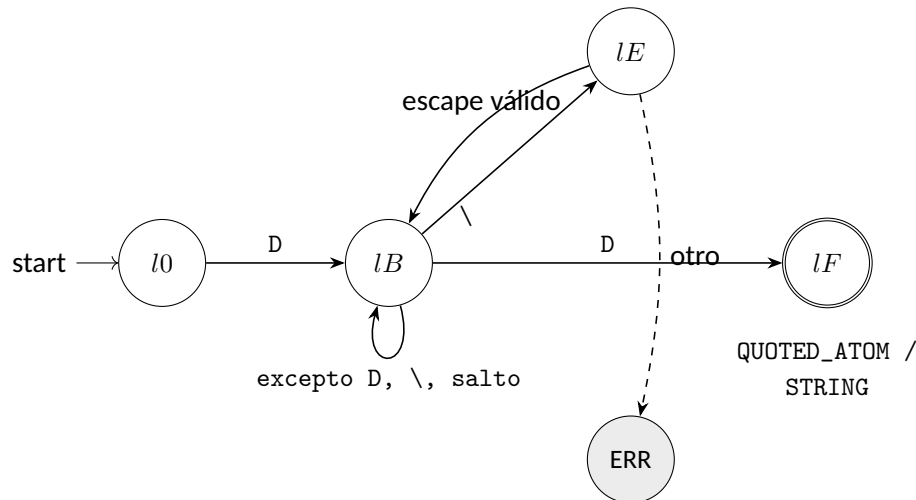


Figura 4: Autómata para literales entrecomillados (átomos citados y cadenas).

Los comentarios separan la rama de línea (% hasta el salto, que acepta de inmediato y cicla hasta el fin de línea) de la rama de bloque (/ * . . . */ hasta el primer cierre). Solo la secuencia */ cierra la rama de bloque; cualquier otro símbolo, incluido un * aislado, mantiene el recorrido dentro del comentario.

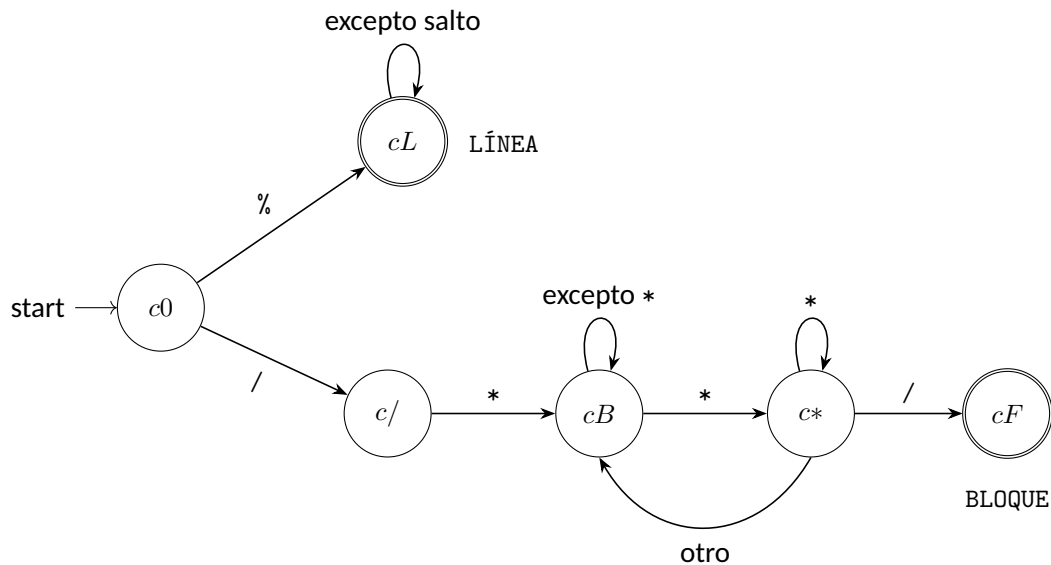


Figura 5: Autómata para comentarios de línea y de bloque.

El trie representativo de operadores comparte prefijos para aplicar máxima coincidencia antes de emitir el token: cada camino desde o0 deletrea el lexema y solo los nodos de doble círculo aceptan, conservando lexema y familia. Por legibilidad se omiten las ramas directas de un carácter (; ! + <), que aceptan directamente desde o0 igual que las dibujadas; la coma no es un operador sino el delimitador COMMA. Las palabras is y mod requieren además la guarda de frontera de palabra.

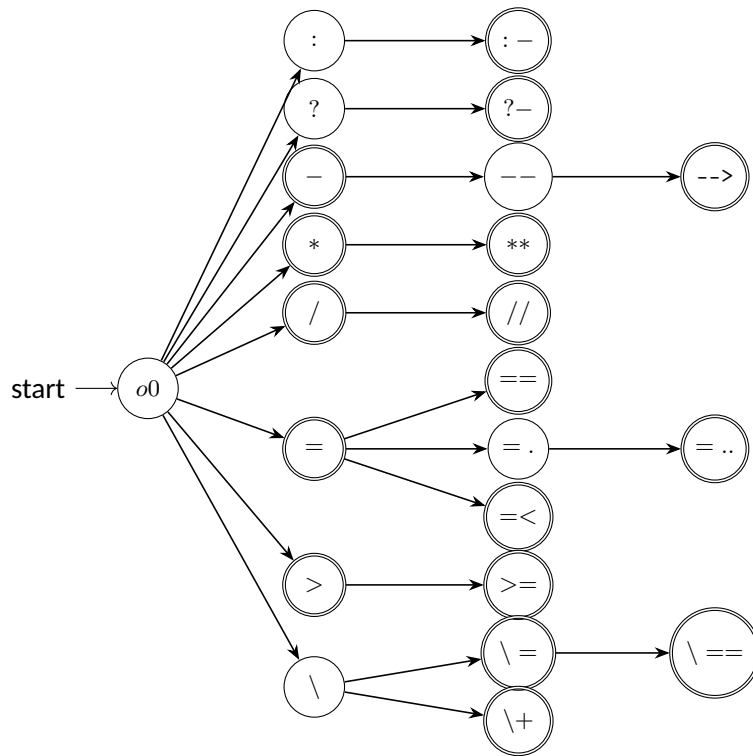


Figura 6: Trie representativo de operadores con prefijos compartidos.

El despachador integrado decide por la primera clase de carácter y luego aplica máxima coincidencia y prioridad, lo que equivale a unir los autómatas por transiciones epsilon, determinar y etiquetar cada aceptación.

Para mostrar el procedimiento completo se tomó el lenguaje representativo $\{=, ==, \backslash=, \backslash==\}$. El AFN une mediante epsilon las ramas $q_0 \rightarrow q_1 = q_2 = q_3$ y $q_0 \rightarrow q_4 \backslash q_5 = q_6 = q_7$; los estados q_2, q_3, q_6 y q_7 aceptan OP_EQ, OP_EQEQ, OP_NEQ y OP_NEQEQ con etiquetas distintas porque el lexer debe producir lexemas distintos.

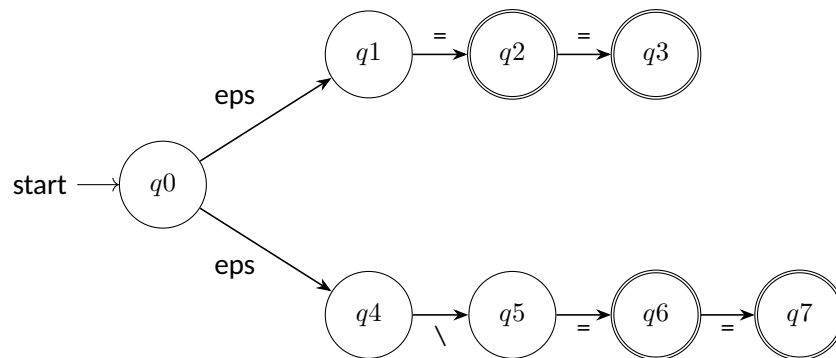


Figura 7: AFN representativo de operadores.

La construcción de subconjuntos comienza con A como clausura epsilon de $\{q_0\} = \{q_0, q_1, q_4\}$. La tabla contiene todos los subconjuntos alcanzables, incluido el estado muerto Z.

Estado	Subconjunto AFN	Con =	Con \	Aceptación
A	{q0, q1, q4}	B	D	ninguna
B	{q2}	C	Z	OP_EQ
C	{q3}	Z	Z	OP_EQEQ
D	{q5}	E	Z	ninguna
E	{q6}	F	Z	OP_NEQ
F	{q7}	Z	Z	OP_NEQEQ
Z	vacío	Z	Z	ninguna

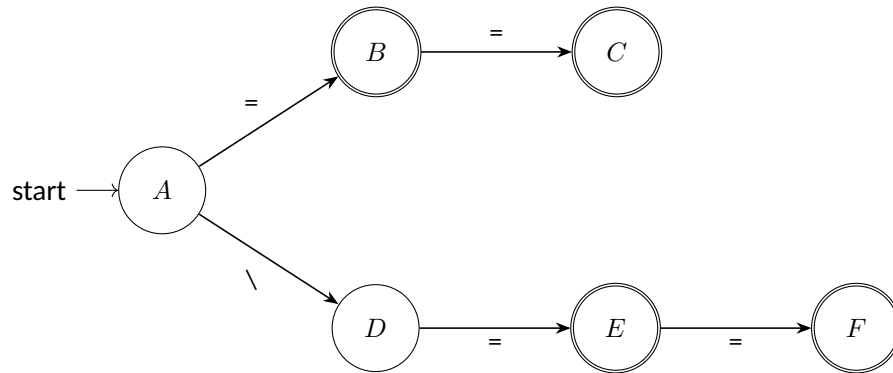


Figura 8: AFD obtenido por construcción de subconjuntos.

Para minimizar el AFD como clasificador de tokens, la partición inicial separa no aceptores y cada salida observable. Las firmas de A, D y Z son diferentes: conducen, respectivamente, a B, E y Z. El bloque se refina a {A}, {D} y {Z}. La partición estable conserva siete estados y las cuatro salidas; ignorar las etiquetas permitiría fusiones incorrectas para un clasificador léxico. La Figura 9 muestra el AFD mínimo completo, con el sumidero Z al que llegan las transiciones de rechazo omitidas en la Figura 8.

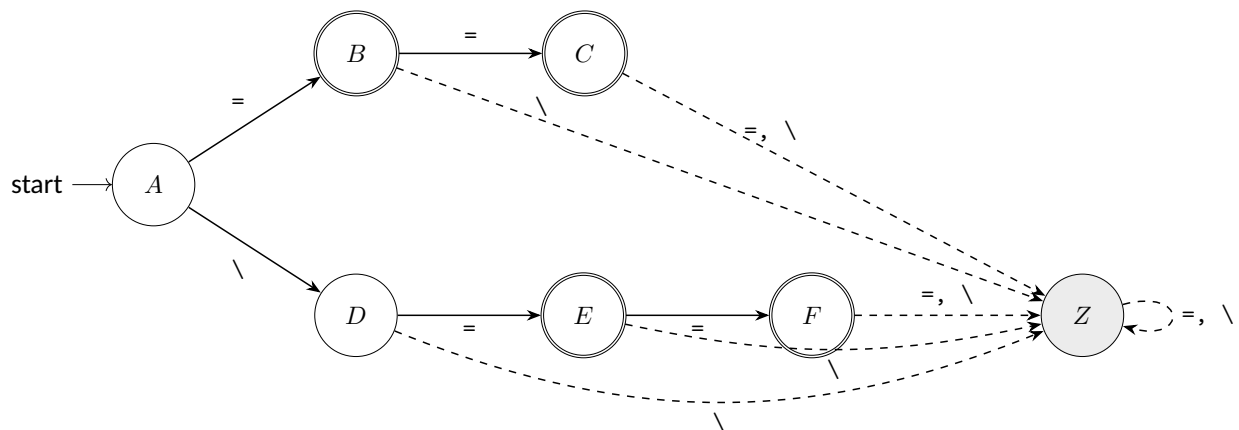


Figura 9: AFD mínimo etiquetado, con estado sumidero.

6. Reglas de prioridad y máxima coincidencia

En cada posición se elige el candidato válido más largo y solo en empate se aplica la prioridad contractual.

Caso	Regla aplicada	Resultado esperado
\== / \= / == (Po1)	Se elige el lexema válido más largo (3 caracteres).	OPERATOR \== de unificación
= . . / = (Po2)	Más largo (3 caracteres) frente a prefijo de 1.	OPERATOR = . . de unificación
== / = (Po3)	Más largo (2 caracteres).	OPERATOR ==
:- / : (Po5)	Operador exacto de dos caracteres; : solo es inválido.	OPERATOR :- de cláusula
?- / ? (Po6)	Operador exacto de dos caracteres; ? solo es inválido.	OPERATOR ?- de consulta
--> / - (Po7)	Más largo (3 caracteres).	OPERATOR --> DCG
// / / (Po8)	Más largo (2 caracteres).	OPERATOR // aritmético
** / * (Po9)	Más largo (2 caracteres).	OPERATOR ** aritmético
_ / _Variable (P10-P11)	El guion solo es anónimo; con continuación gana la variable más larga.	ANONYMOUS_VARIABLE o VARIABLE
is / isla, mod / modulo (P12-P15)	Palabra reservada solo sin continuación; si hay continuación gana el átomo más largo.	OPERATOR u ATOM según frontera
1.25 (P16)	REAL más largo frente a prefijo INTEGER.	REAL
12abc (P21)	Número con sufijo inválido: un único error hasta separador.	MALFORMED_NUMBER, sin tokens parciales
5., .5, 1..2 (Io7-Io9)	Detector numérico antes que DOT; no se emiten parciales.	Error, sin tokens parciales

Las palabras `is` y `mod` solo ganan cuando no tienen continuación de identificador; así, `is_` y `modulo` son átomos. El caso `12abc` consume un único fragmento numérico inválido y `X=Y` produce tres tokens adyacentes porque el cursor avanza sin exigir separadores.

7. Diseño e implementación

7.1. Arquitectura de la solución

La arquitectura tiene cinco módulos con una sola dirección de dependencias: `tokens.py` define `TokenType`, `OperatorFamily`, el mapa único de operadores y el objeto `Token`; `errors.py` define el tipo de error y el diagnóstico; `symbol_table.py` implementa la tabla con deduplicación por par tipo y lexema; `lexer.py` expone el lexer y las funciones de lectura; y `main.py` ofrece la CLI. La validación numérica usa coincidencia completa sobre el fragmento candidato (Python Software Foundation, 2026).

El flujo recorre la entrada de izquierda a derecha con un cursor de índice, línea y columna: primero consume ignorables, luego literales, números, identificadores, operadores por longitud descendente, delimitadores y finalmente el carácter inválido. Cada token guarda la línea y columna de su primer carácter; la tabulación avanza una columna y CRLF cuenta como un único salto.

7.2. Formato de salida

```
<TIPO_TOKEN, 'lexema', linea, columna>
```

Un fragmento real de la salida generada por el programa es:

```

<ATOM, 'padre', 2, 1>
<LPAREN, '(', 2, 6>
<ATOM, 'juan', 2, 7>
<COMMA, ',', 2, 11>
<ATOM, 'ana', 2, 13>
<RPAREN, ')', 2, 16>
<DOT, '.', 2, 17>

```

El archivo completo válido produce 207 tokens, 0 errores y 48 entradas de tabla. El programa con errores produce 69 tokens recuperados y 12 diagnósticos. Los comandos reproducibles son:

```

python -m compileall -q src tests tools
uv run --with pytest python -m pytest -q
python -m src.main tests/corpus/entradas/programa_valido.pl
python -m src.main tests/corpus/entradas/programa_con_errores.pl

```

7.3. Tabla de lexemas

Se registran ATOM, QUOTED_ATOM y VARIABLE como identificadores, más INTEGER, REAL y STRING como literales conservables, siempre con el lexema original y sin conversiones. La clave de deduplicación es el par tipo y lexema. La variable anónima _ nunca se registra porque cada aparición es independiente; operadores, delimitadores, espacios y comentarios tampoco.

Por ejemplo, para la entrada de dos líneas `padre(juan, ana) . repetidas`, la salida con `--show-attributes` agrega el índice de tabla a cada token conservable:

```

<ATOM, 'padre', 1, 1, 0>
<ATOM, 'juan', 1, 7, 1>
<ATOM, 'padre', 2, 1, 0>
<ATOM, 'juan', 2, 7, 1>

```

y la opción `--tabla` vuelca las entradas deduplicadas:

Índice	Tipo	Lexema
0	ATOM	padre
1	ATOM	juan
2	ATOM	ana

El token `padre` de la línea 2 reutiliza la entrada 0: dos tokens apuntan a la misma entrada y no se crea un duplicado.

8. Tratamiento de errores léxicos

Error	Ejemplo	Mensaje producido	Recuperación
Átomo sin cierre	' Juan	átomo sin cierre	descarta hasta antes del salto o hasta EOF
Cadena sin cierre	" Juan	cadena sin cierre	descarta hasta antes del salto o hasta EOF
Comentario sin cierre	/* texto	bloque sin cierre	consume hasta EOF
Carácter no admitido	@	carácter no admitido	consume un carácter y continúa
Número mal formado	1..2	número mal formado	consume la corrida completa de dígitos y puntos más sufijo
Escape inválido	'a\q'	escape desconocido	consume la pareja y sigue buscando el cierre

Los seis errores se acumulan sin detener el recorrido y cada diagnóstico incluye clase, fragmento, línea, columna y mensaje útil. Un escape desconocido registra el error, consume la pareja y continúa buscando el cierre. La salida real del archivo con errores contiene exactamente 12 diagnósticos, con tokens válidos después de cada error recuperable y ningún token después del bloque sin cierre final.

El caso 5. ilustra la prioridad del reconocedor numérico: el analizador consume primero la corrida máxima de dígitos y puntos y solo después decide; si la corrida no es `INTEGER` ni `REAL` se emite `MALFORMED_NUMBER` sin tokens parciales, en lugar de `INTEGER(5)` seguido de `DOT`. Esta prioridad del detector de error por sobre el tokenizado normal es una convención documentada del contrato: evita partir flotantes (3.14 nunca se divide), hace el diagnóstico determinista y obliga a separar con espacio el punto final tras un número (5.). La máxima coincidencia sigue rigiendo entre tokens válidos; el reconocedor de error solo prevalece cuando ningún token válido cubre la corrida completa.

9. Plan de pruebas y resultados

ID	Entrada o archivo	Objetivo	Resultado esperado	Resultado obtenido	Estado
V23	padre(juan, ana).	Hecho y posiciones	7 tokens en columnas exactas	7 tokens en columnas exactas	OK
P01	\==	Máxima coincidencia	Operador completo	Operador completo de unificación	OK
P21	12abc	Recuperación numérica	Un error, sin parciales	Un error, sin tokens parciales	OK
I02	Átomo sin cierre + línea válida	Recuperación tras salto	Error y tokens en línea 2	Error y tokens en línea 2	OK
F01	programa_valido.pl	Cobertura integral	207 tokens, 0 errores	207 tokens, 0 errores, 17 tipos y 6 familias	OK
F02	programa_con_errores.pl	Diagnósticos y continuidad	69 tokens y 12 errores	69 tokens, 12 errores y 28 entradas	OK

La suite `tests/test_lexer.py` suma 86 pruebas: 3 del modelo `Token`, 27 válidas `V01-V27`, 22 de prioridad `P01-P22`, 16 inválidas `I01-I16`, 12 transversales `T01-T12` y 6 de archivos `F01-F06`. La matriz completa queda disponible en `docs/informe/04_matriz_de_pruebas.md` y el corpus en `tests/corpus/`.

9.1. Análisis de resultados

El archivo válido produce 207 tokens, 0 errores y 48 entradas, cubriendo los 17 tipos de token y las 6 familias. El archivo con errores produce 69 tokens, 12 errores de las 6 clases en 12 líneas distintas y 28 entradas, con tokens válidos después de cada error recuperable y ningún token después del bloque sin cierre final. El comando ejecutado fue `uv run --with pytest python -m pytest -q`, con resultado real de 86 aprobadas (pytest-dev team, 2026); además `python -m compileall src tests` terminó sin errores y `git diff --check` salió limpio.

La cobertura incluye cierres a EOF y antes de salto, escapes en ambos delimitadores, cinco formas numéricas mal formadas, caracteres aislados, CRLF, tabulación, comentarios dentro de literales, -12 como dos tokens y) aislado como token. La dificultad principal fue distinguir el punto de fin de cláusula del punto decimal: una cláusula que termina en número sin espacio (`mod 5.`) es número mal formado, por lo que los archivos válidos separan el número del punto final.

10. Conclusiones

El trabajo muestra que un lexer riguroso nace de decisiones contractuales explícitas antes que del código: alfabeto ASCII, números sin signo, escapes cerrados por delimitador, comentarios no anidables y máxima coincidencia con prioridades resolvieron las ambigüedades sin casos especiales en la implementación.

Los objetivos se cumplieron de forma medible con 86 pruebas aprobadas, cobertura total de categorías y familias, y dos archivos completos que ejercitan el recorrido real. La separación entre especificación, implementación y validación permite rastrear cada regla desde su expresión regular hasta su prueba.

Las limitaciones son que el lexer solo reconoce el subconjunto contratado y que los archivos válidos deben separar con espacio un número del punto final por la regla de número mal formado. Como mejoras futuras se proponen modos de salida adicionales, un visualizador de autómatas generado desde las expresiones del código y pruebas de rendimiento sobre archivos grandes, sin ampliar el subconjunto ni entrar en análisis sintáctico.

11. Referencias

- Lévano, M. (2026). *Tarea: análisis léxico del lenguaje Prolog*. Investigación, análisis, diseño, implementación y validación (INFO1148, sem2_2026). Enunciado del curso.
- ISO/IEC. (1995). *ISO/IEC 13211-1:1995. Information technology—Programming languages—Prolog—Part 1: General core*.
- Python Software Foundation. (2026). *Regular expression operations (re)*. <https://docs.python.org/3/library/re.html>
- pytest-dev team. (2026). *pytest documentation*. <https://docs.pytest.org/>
- Astral Software. (2026). *uv documentation*. <https://docs.astral.sh/uv/>
- SWI-Prolog. (s. f.). *SWI-Prolog documentation*. <https://www.swi-prolog.org/>
- Equipo de la asignatura INFO1148. (2026). *Formato Informe Tarea*. Archivo local Formato Informe Tarea.pdf. Directorio: docs/referencias/.
- Equipo de trabajo. (2026). *Especificación léxica contractual, matriz de pruebas y código del analizador*. Repositorio <https://github.com/FernandoValdes01/Tarea01TC>.

12. Anexos

Recurso	Enlace
Repositorio Git	https://github.com/FernandoValdes01/Tarea01TC

12.1. Anexo A. Evidencias adicionales

El contenido canónico del informe está en docs/informe/Tarea_JefeGrupo_FernandoValdes.tex. Los nombres, fecha, autoría y resultados corresponden al trabajo de Fernando Alonso Valdés Gómez, Juan Francisco Muñoz Veloso y Vicente Rodrigo Rivera Sánchez.

La especificación contractual vive en docs/informe/01_especificacion_lexica.md y la matriz completa en docs/informe/04_matriz_de_pruebas.md. El código está en src/ y el corpus en tests/corpus/.

Comandos ejecutados desde la raíz del repositorio:

```
python -m compileall -q src tests tools
uv run --with pytest python -m pytest -q
python -m src.main tests/corpus/entradas/programa_valido.pl
```

```
python -m src.main tests/corpus/entradas/programa_con_errores.pl  
git diff --check
```

Resultados reales: `compileall` sin errores, 86 pruebas aprobadas, PDF generado correctamente y `git diff --check` limpio. El archivo válido produce 207 tokens, 0 errores y 48 entradas; el archivo con errores produce 69 tokens, 12 errores y 28 entradas. Frente a la rúbrica: especificación y expresiones regulares en la sección 4, autómatas con AFN, subconjuntos y minimización en la sección 5, implementación con posiciones y tabla en la sección 7, y pruebas con errores léxicos en las secciones 8 y 9.

Los diagramas oficiales del trabajo son los de la sección 5. Sus fuentes editables en Mermaid se conservan en `docs/automatas/diagramas/` y el desarrollo formal de cada construcción en `docs/automatas/`.